데이터 전처리

COW AI팀 데이터사이언스트랙 2조

데이터 둘러보기

데이터 구조 간단하게 살펴보기

```
1 import pandas as pd
2
3 df = pd.read_csv('/content/shopping_behavior_updated.csv')
4
5 display(df.head())
```

index	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring

Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
3.1	Yes	Express	Yes	Yes	14	Venmo	Fortnightly
3.1	Yes	Express	Yes	Yes	2	Cash	Fortnightly
3.1	Yes	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3.5	Yes	Next Day Air	Yes	Yes	49	PayPal	Weekly
2.7	Yes	Free Shipping	Yes	Yes	31	PayPal	Annually

데이터가 가지고있는 열을 모두 출력해보기

```
1 for column in df.columns:
2     print(column)
```

```
Customer ID
Age
Gender
Item Purchased
Category
Purchase Amount (USD)
Location
Size
Color
Season
Review Rating
Subscription Status
Shipping Type
Discount Applied
Promo Code Used
Previous Purchases
Payment Method
Frequency of Purchases
```

Customer ID - 고객 식별 번호
Age - 고객 나이
Gender - 고객 성별
Item Purchased - 구매한 상품명
Category - 상품 카테고리
Purchase Amount (USD) - 구매 금액(달러)
Location - 고객의 지역
Size - 구매 상품의 크기
Color - 구매 상품의 색상
Season - 구매 시점의 계절
Review Rating - 상품 리뷰 평점
Subscription Status - 구독 서비스 가입 여부
Shipping Type - 배송 방식
Discount Applied - 할인 적용 여부
Promo Code Used - 프로모션 코드 사용 여부
Previous Purchases - 이전 구매 횟수
Payment Method - 결제 수단
Frequency of Purchases - 구매 빈도

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3900 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
None
```

```
print("=== 데이터 정보 ===")
print(df.info())
```

데이터 정보

object - 문자열 타입이거나 여러 타입이 섞여있는 타입

```
=== 컬럼별 결측치 개수 ===
Customer ID                0
Age                        0
Gender                     0
Item Purchased             0
Category                   0
Purchase Amount (USD)      0
Location                   0
Size                       0
Color                      0
Season                     0
Review Rating              0
Subscription Status        0
Shipping Type              0
Discount Applied           0
Promo Code Used            0
Previous Purchases         0
Payment Method             0
Frequency of Purchases     0
dtype: int64
```

```
print("₩n=== 컬럼별 결측치 개수 ===")
print(df.isnull().sum())
```

각 열들 모두 결측치가 없는 걸 확인할 수 있음

```python
duplicates = df.duplicated().sum()
print(f"중복된 행의 개수: {duplicates}")
```

```
중복된 행의 개수: 0
```

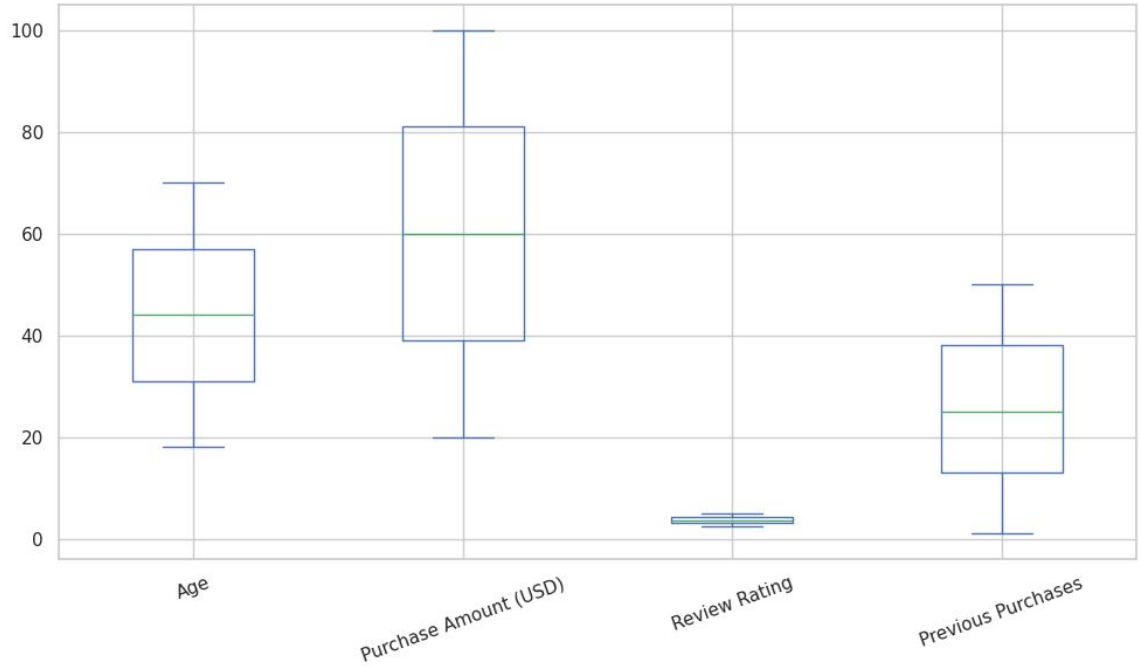
중복된 행은 보이지 않음

```python
numeric_columns = ["Age", "Purchase Amount (USD)", "Review Rating", "Previous Purchases"]

df[numeric_columns].plot(kind="box", figsize=(10, 6))
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

이상치 확인은 박스플롯을 찍어서 확인해보기

수염을 벗어나는 큰 이상치는 보이지 않음



```
print("₩n=== 수치형 데이터 기초 통계량 ===")
display(df.describe())
```

수치형 데이터들의 기초 통계량

	Customer ID	Age	Purchase Amount (USD)	Review Rating	Previous Purchases
count	3900.000000	3900.000000	3900.000000	3900.000000	3900.000000
mean	1950.500000	44.068462	59.764359	3.749949	25.351538
std	1125.977353	15.207589	23.685392	0.716223	14.447125
min	1.000000	18.000000	20.000000	2.500000	1.000000
25%	975.750000	31.000000	39.000000	3.100000	13.000000
50%	1950.500000	44.000000	60.000000	3.700000	25.000000
75%	2925.250000	57.000000	81.000000	4.400000	38.000000
max	3900.000000	70.000000	100.000000	5.000000	50.000000

Customer ID는 필요없어보이는 정보. 전처리 필요해보임

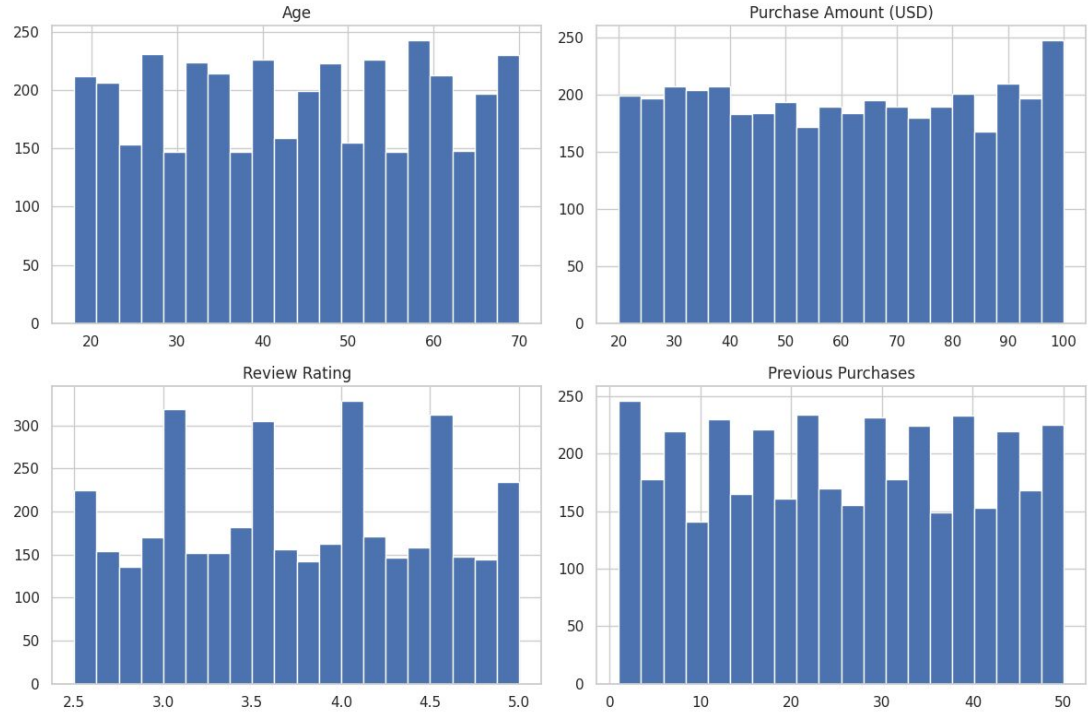
나머지 수치형 데이터들 끼리도 스케일의 차이가 심함. 값들을 표준화해줘야 군집화가 잘 되니까 이 점도 전처리 시 고려해야할 점

```
numeric_columns = ["Age", "Purchase Amount (USD)", "Review Rating", "Previous Purchases"]

df[numeric_columns].hist(figsize=(12, 8), bins=20)

plt.tight_layout()
plt.show()
```

수치화 데이터들을 시각화(20개의 구간으로 끊어서)

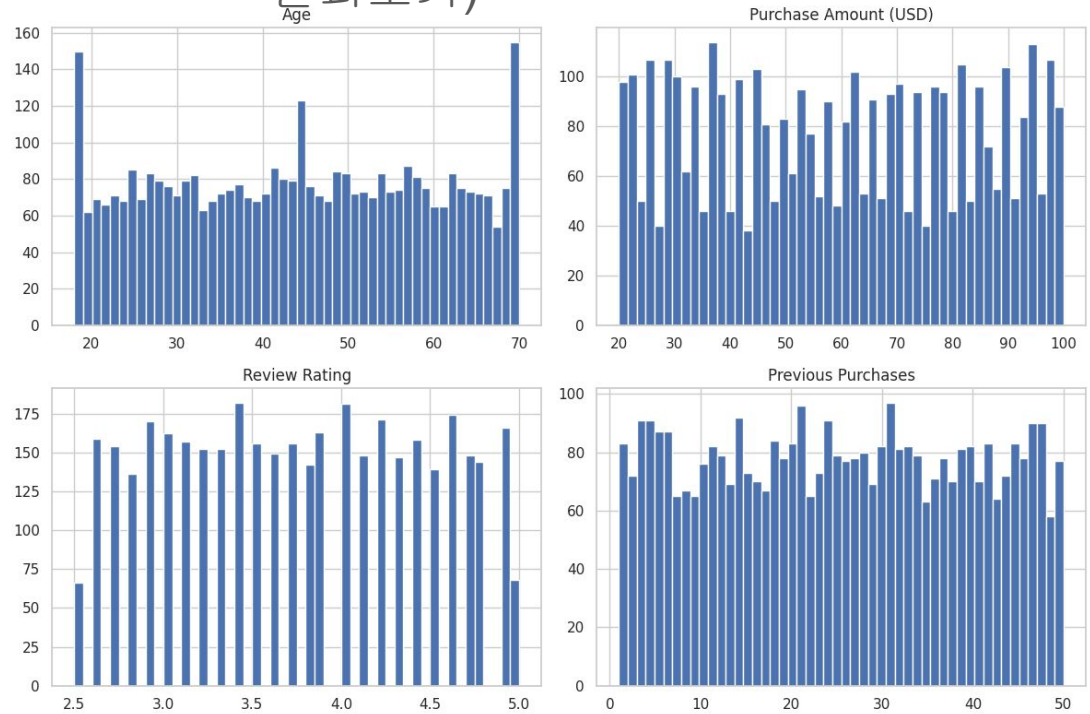


```python
numeric_columns = ["Age", "Purchase Amount (USD)", "Review Rating", "Previous Purchases"]

df[numeric_columns].hist(figsize=(12, 8), bins=50)

plt.tight_layout()
plt.show()
```

수치화 데이터들을 시각화(50개의 구간으로 끊어서 더 자세하게 살펴보기)



```python
categorical_cols = ['Gender', 'Category', 'Size', 'Season', 'Subscription Status', 'Discount Applied', 'Promo Code Used', 'Payment Method', 'Frequency of Purchases']

print("₩n=== 범주형 변수별 고유값 확인 ===")
for col in categorical_cols:
    print(f"[{col}]: {df[col].unique()}")
```

```
=== 범주형 변수별 고유값 확인 ===
[Gender]: ['Male' 'Female']
[Category]: ['Clothing' 'Footwear' 'Outerwear' 'Accessories']
[Size]: ['L' 'S' 'M' 'XL']
[Season]: ['Winter' 'Spring' 'Summer' 'Fall']
[Subscription Status]: ['Yes' 'No']
[Discount Applied]: ['Yes' 'No']
[Promo Code Used]: ['Yes' 'No']
[Payment Method]: ['Venmo' 'Cash' 'Credit Card' 'PayPal' 'Bank Transfer' 'Debit Card']
[Frequency of Purchases]: ['Fortnightly' 'Weekly' 'Annually' 'Quarterly' 'Bi-Weekly' 'Monthly'
 'Every 3 Months']
```

대표적인 범주형 변수들은 위와 같은 값들을 가짐

```python
categorical_columns = df.select_dtypes(include="object").columns

for column in categorical_columns:
    print(f"{column} ({df[column].nunique()}개)")
    print(df[column].unique())
    print()
```

범주형 데이터들 (object 타입)의 모든 값들을 출력해봄

```
Gender (2개)
['Male' 'Female']

Item Purchased (25개)
['Blouse' 'Sweater' 'Jeans' 'Sandals' 'Sneakers' 'Shirt' 'Shorts' 'Coat'
 'Handbag' 'Shoes' 'Dress' 'Skirt' 'Sunglasses' 'Pants' 'Jacket' 'Hoodie'
 'Jewelry' 'T-shirt' 'Scarf' 'Hat' 'Socks' 'Backpack' 'Belt' 'Boots'
 'Gloves']

Color (25개)
['Gray' 'Maroon' 'Turquoise' 'White' 'Charcoal' 'Silver' 'Pink' 'Purple'
 'Olive' 'Gold' 'Violet' 'Teal' 'Lavender' 'Black' 'Green' 'Peach' 'Red'
 'Cyan' 'Brown' 'Beige' 'Orange' 'Indigo' 'Yellow' 'Magenta' 'Blue']

Season (4개)
['Winter' 'Spring' 'Summer' 'Fall']

Subscription Status (2개)
['Yes' 'No']

Shipping Type (6개)
['Express' 'Free Shipping' 'Next Day Air' 'Standard' '2-Day Shipping'
 'Store Pickup']

Category (4개)
['Clothing' 'Footwear' 'Outerwear' 'Accessories']

Location (50개)
['Kentucky' 'Maine' 'Massachusetts' 'Rhode Island' 'Oregon' 'Wyoming'
 'Montana' 'Louisiana' 'West Virginia' 'Missouri' 'Arkansas' 'Hawaii'
 'Delaware' 'New Hampshire' 'New York' 'Alabama' 'Mississippi'
 'North Carolina' 'California' 'Oklahoma' 'Florida' 'Texas' 'Nevada'
 'Kansas' 'Colorado' 'North Dakota' 'Illinois' 'Indiana' 'Arizona'
 'Alaska' 'Tennessee' 'Ohio' 'New Jersey' 'Maryland' 'Vermont'
 'New Mexico' 'South Carolina' 'Idaho' 'Pennsylvania' 'Connecticut' 'Utah'
 'Virginia' 'Georgia' 'Nebraska' 'Iowa' 'South Dakota' 'Minnesota'
 'Washington' 'Wisconsin' 'Michigan']

Size (4개)
['L' 'S' 'M' 'XL']

Color (25개)
['Gray' 'Maroon' 'Turquoise' 'White' 'Charcoal' 'Silver' 'Pink' 'Purple'
 'Olive' 'Gold' 'Violet' 'Teal' 'Lavender' 'Black' 'Green' 'Peach' 'Red'
 'Cyan' 'Brown' 'Beige' 'Orange' 'Indigo' 'Yellow' 'Magenta' 'Blue']

Discount Applied (2개)
['Yes' 'No']

Promo Code Used (2개)
['Yes' 'No']

Payment Method (6개)
['Venmo' 'Cash' 'Credit Card' 'PayPal' 'Bank Transfer' 'Debit Card']

Frequency of Purchases (7개)
['Fortnightly' 'Weekly' 'Annually' 'Quarterly' 'Bi-Weekly' 'Monthly'
 'Every 3 Months']
```

```python
def categorize_age(age):
    if age < 20:
        return '10s'
    elif age < 30:
        return '20s'
    elif age < 40:
        return '30s'
    elif age < 50:
        return '40s'
    elif age < 60:
        return '50s'
    else:
        return '60s+'

df['Age Group'] = df['Age'].apply(categorize_age)

print("=== 연령대별 데이터 분포 ===")
print(df['Age Group'].value_counts().sort_index())

display(df[['Age', 'Age Group']].head())
```

```
=== 연령대별 데이터 분포 ===
Age Group
10s     150
20s     728
30s     724
40s     739
50s     771
60s+    788
Name: count, dtype: int64
```

수치화로 쪼개져있는 Age를 10대씩 그룹화해서 다시 측정해봄

10대가 가장 적고 20~60+의 데이터에는 고루 퍼져있음. 60+가 가장 많은 걸 볼 수 있다. 10대가 가장 적은 이유는 Age의 분포가 18 ~ 70이라서 그런

```python
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
plt.rcParams['font.family'] = 'sans-serif'

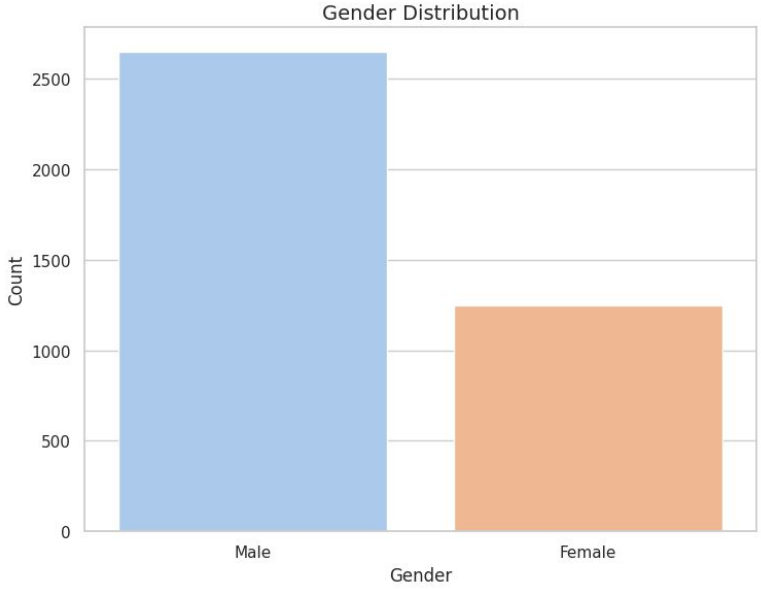
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

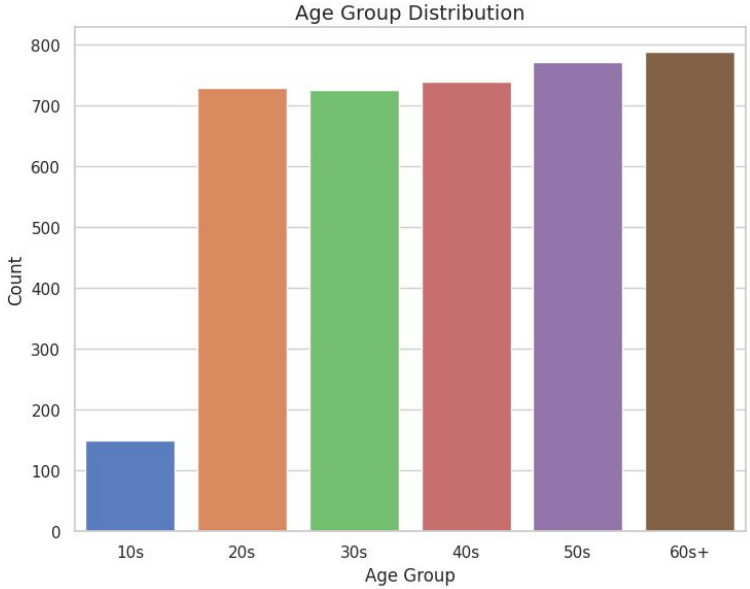
sns.countplot(data=df, x='Gender', ax=axes[0], palette='pastel')
axes[0].set_title('Gender Distribution', fontsize=14)
axes[0].set_xlabel('Gender')
axes[0].set_ylabel('Count')

sns.countplot(data=df, x='Age Group', ax=axes[1], order=['10s', '20s', '30s', '40s', '50s', '60s+'], palette='muted')
axes[1].set_title('Age Group Distribution', fontsize=14)
axes[1].set_xlabel('Age Group')
axes[1].set_ylabel('Count')

plt.tight_layout()
plt.show()
```

연령은 18~70세에 걸쳐 전반적으로 분포하며, 연령대 구간의 너비가 달라 단순 인원 비교에는 주의가 필요해보임. 전처리할 때 고려해야할점. 10대는 18, 19세 밖에 없어서 10대를 따로 둬야할지 아니면 18~29처럼 분류해야할지 생각해봐야할 것 같음..





```
1 print(pd.crosstab(df["Discount Applied"], df["Promo Code Used"]))
```

Promo Code Used	No	Yes
Discount Applied		
No	2223	0
Yes	0	1677

Discount Applied랑 Promo Code Used의 값들이 겹치지 않는 것 같아서 비교해봤더니
두 열이 완전히 똑같이 움직이는 것으로 확인됨. 둘 다 사용할 필요 없이 하나는 제거해도 될 것 같음 -> 전처리 고려

```
binary_columns = ["Subscription Status", "Discount Applied", "Promo Code Used"]

for column in binary_columns:
    df[column] = df[column].map({"No": 0, "Yes": 1})
```

```
df["Gender"] = df["Gender"].map({"Female": 0, "Male": 1})
```

으로 범주형 데이터들을 수치화하는 작업이 필요해보임
Subscription Status랑 Discount Applided, Promo Code Used는 No 아님 Yes 뿐이니까 0과 1로 치환. Gender도 Female과 Male을 0과 1로 치환. 0과 1은 크기 비교가 아니라 그냥 숫자로 바꾼 값들

```
df_encoded = pd.get_dummies(df, columns=["Category"], dtype=int)

df_encoded.head()
```

lor	Season	Review Rating	...	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases	Category_Accessories	Category_Clothing	Category_Footwear	Category_Outerwear
ray	Winter	3.1	...	Express	1	1	14	Venmo	Fortnightly	0	1	0	0
oon	Winter	3.1	...	Express	1	1	2	Cash	Fortnightly	0	1	0	0
oon	Spring	3.1	...	Free Shipping	1	1	23	Credit Card	Weekly	0	1	0	0
oon	Spring	3.5	...	Next Day Air	1	1	49	PayPal	Weekly	0	0	1	0
oise	Spring	2.7	...	Free Shipping	1	1	31	PayPal	Annually	0	1	0	0

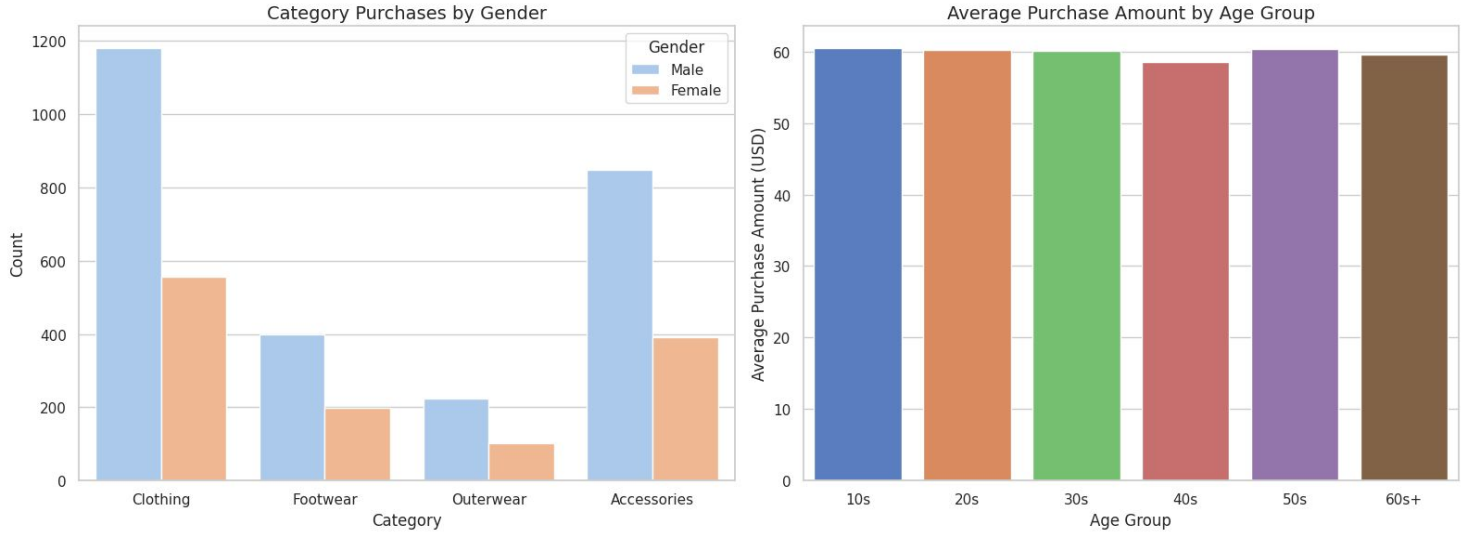
Size, Category, Season, Payment Method, Frequency of Purchases처럼 이진화 할 수 없는 값들은 원-핫 인코딩으로 표현 가능함. 위의 이미지는 Category에 대한 원-핫 인코딩 예제

```python
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# 성별에 따른 카테고리별 구매 빈도
sns.countplot(data=df, x='Category', hue='Gender', ax=axes[0], palette='pastel')
axes[0].set_title('Category Purchases by Gender', fontsize=14)
axes[0].set_xlabel('Category')
axes[0].set_ylabel('Count')

# 연령대별 평균 구매 금액
age_purchase = df.groupby('Age Group')['Purchase Amount (USD)'].mean().reset_index()
sns.barplot(data=age_purchase, x='Age Group', y='Purchase Amount (USD)', ax=axes[1], order=['10s', '20s', '30s', '40s', '50s', '60s+'], hue='Age Group', legend=False, palette='muted')
axes[1].set_title('Average Purchase Amount by Age Group', fontsize=14)
axes[1].set_xlabel('Age Group')
axes[1].set_ylabel('Average Purchase Amount (USD)')

plt.tight_layout()
plt.show()
```



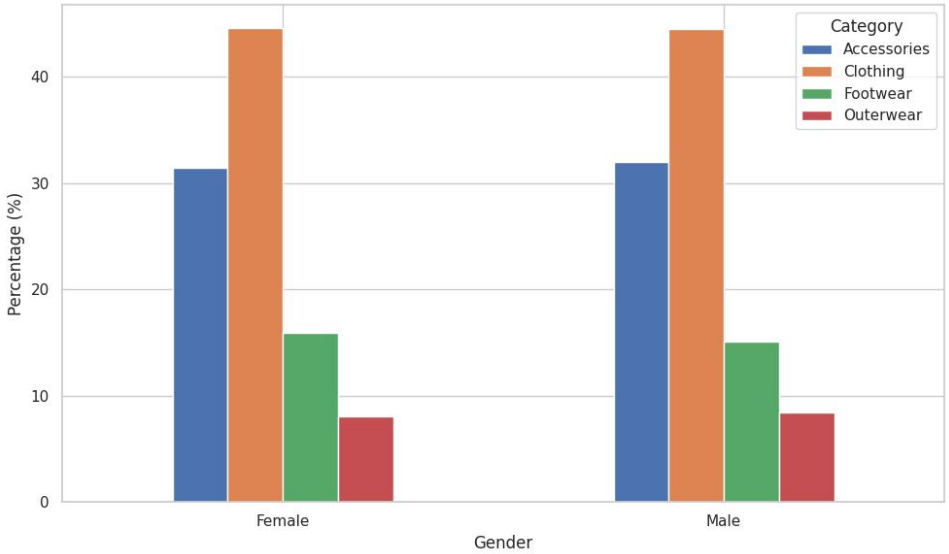
성별에 따른 카테고리별 구매 빈도를 보면 모든 곳에서 남성이 높은 게 보이는데 아마 남성의 데이터셋에 많아서 생긴 걸로 보임

```python
category_ratio = pd.crosstab(df["Gender"], df["Category"], normalize="index") * 100

print(category_ratio)

category_ratio.plot(kind="bar", figsize=(10, 6))

plt.ylabel("Percentage (%)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



그래서 남녀에 따른 카테고리 분포를
개수가 아니라 비율로 다시
시각화해봄

전처리 시작

```
1 from sklearn.preprocessing import StandardScaler
2
3 df = pd.read_csv("shopping_behavior_updated.csv")
4
5 df.shape
```

```
(3900, 18)
```

전처리를 시작하기 위해서 데이터를 다시 가져옴

```
1 drop_columns = ["Customer ID", "Discount Applied"]
2
3 df = df.drop(columns=drop_columns)
4
5 df.shape
```

```
(3900, 16)
```

일단 의미가 없는 Customer ID와 Promo Code Used와 중복되는 의미를 가진 Discount Applied를 제거함

```python
frequency_map = {
    "Weekly": 365 / 7,
    "Bi-Weekly": 365 / 14,
    "Fortnightly": 365 / 14,
    "Monthly": 12,
    "Every 3 Months": 4,
    "Quarterly": 4,
    "Annually": 1
}

df["Purchase Frequency Per Year"] = df["Frequency of Purchases"].map(frequency_map)
df = df.drop(columns=["Frequency of Purchases"])

df["Purchase Frequency Per Year"].head(10)
```

	Purchase Frequency Per Year
0	26.071429
1	26.071429
2	52.142857
3	52.142857
4	1.000000
5	52.142857
6	4.000000
7	52.142857
8	1.000000
9	4.000000

dtype: float64

구매 빈도를 연간 예상 구매 횟수로 변환. 예를 들어서 Weekly는 52회/년, Monthly는 12회/년, Annually는 1회/년으로 지정해서 저장시킴. 이렇게 저장한 이유는 숫자가 클수록 구매를 자주 하는 고객이 되니까 군집 결과를 해석하기 편할 것 같아서

이걸 원-핫 인코딩으로 할지 아니면 저렇게 할지 고민했는데 일단은 저렇게 전처리하고 나중에 원-핫 인코딩을 진행하든지 처리할 방법이 있을듯.. 7주차 성능 향상 시간 때 생각해볼 부분인듯

```python
numeric_columns = df.select_dtypes(include=["int64", "float64"]).columns
categorical_columns = df.select_dtypes(include=["object"]).columns

print("수치형 변수")
for col in numeric_columns:
    print(col)

print("₩n범주형 변수")
for col in categorical_columns:
    print(col)
```

```
수치형 변수
Age
Purchase Amount (USD)
Review Rating
Previous Purchases
Purchase Frequency Per Year

범주형 변수
Gender
Item Purchased
Category
Location
Size
Color
Season
Subscription Status
Shipping Type
Discount Applied
Payment Method
```

전처리에 앞서 수치형 데이터와
범주형 데이터를 구분하는
리스트를 하나 만듦

```
scaler = StandardScaler()
scaled_numeric_data = scaler.fit_transform(df[numeric_columns])

scaled_numeric_df = pd.DataFrame(
    scaled_numeric_data,
    columns=numeric_columns,
    index=df.index
)

scaled_numeric_df.describe()
```

수치형 데이터는 StandardScaler를 사용해서 표준화해줌. 표준화 말고 정규화같은 방법도 7주차 때 적용해보면서 실험할 부분으로 생각해야할듯

	Age	Purchase Amount (USD)	Review Rating	Previous Purchases
count	3.900000e+03	3.900000e+03	3.900000e+03	3.900000e+03
mean	-1.985876e-16	-1.166019e-16	4.463666e-16	7.287618e-17
std	1.000128e+00	1.000128e+00	1.000128e+00	1.000128e+00
min	-1.714394e+00	-1.679071e+00	-1.745419e+00	-1.685779e+00
25%	-8.594483e-01	-8.767861e-01	-9.075836e-01	-8.550575e-01
50%	-4.502378e-03	9.950067e-03	-6.974802e-02	-2.433588e-02
75%	8.504436e-01	8.966862e-01	9.077268e-01	8.756125e-01
max	1.705390e+00	1.698971e+00	1.745562e+00	1.706334e+00

$$z = \frac{x_i - \mu}{\sigma}$$

Standardization 식

```
encoded_categorical_df = pd.get_dummies(df[categorical_columns], dtype=int)

encoded_categorical_df.head()
```

	Gender_Female	Gender_Male	Item Purchased_Backpack	Item Purchased_Belt	Item Purchased_Blouse	Item Purchased_Boots	Item Purchased_Coat	Item Purchased_Dress	Purc
0	0	1	0	0	1	0	0	0	
1	0	1	0	0	0	0	0	0	
2	0	1	0	0	0	0	0	0	
3	0	1	0	0	0	0	0	0	
4	0	1	0	0	1	0	0	0	

범주형 데이터들은 원-핫 인코딩으로 수치화해줌

이것도 원-핫 인코딩 말고 다른 방법(레이블 인코딩, 빈도 인코딩 등등...)들은 7주차 때 고민해야할부분일 것 같다. 아예 임베딩을 해버리는 것도 방법일듯

원-핫 인코딩(One-Hot Encoding)은 문자 형태의 범주형 데이터를 여러 개의 0과 1로 이루어진 열로 변환하는 방법. 각 범주마다 새로운 열을 만들고, 해당 범주에 속하면 1, 아니면 0으로 표현함

```
1 print("전처리 후 데이터 크기:", processed_df.shape)
```

```
전처리 후 데이터 크기: (3900, 135)
```

```
1 processed_df.head()
```

	Age	Purchase Amount (USD)	Review Rating	Previous Purchases	Purchase Frequency Per Year	Gender_Female	Gender_Male	Item Purchased_Backpack	Item Purchased_Belt	Item Purchased_Blouse	...	Shipping Type_Standard	Shipping Type_Store Pickup
0	0.718913	-0.285629	-0.907584	-0.785831	0.507793	0	1	0	0	1	...	0	0
1	-1.648629	0.178852	-0.907584	-1.616552	0.507793	0	1	0	0	0	...	0	0
2	0.390088	0.558882	-0.907584	-0.162789	2.054308	0	1	0	0	0	...	0	0
3	-1.517099	1.276716	-0.349027	1.637107	2.054308	0	1	0	0	0	...	0	0
4	0.061263	-0.454531	-1.466141	0.391025	-0.979404	0	1	0	0	1	...	0	0

5 rows × 135 columns

수치화 데이터들을 표준화한 데이터 프레임과 범주형 데이터를 원-핫 인코딩한 데이터를 합치고 결과를 확인

```python
for col in processed_df.columns.tolist():
  print(col)
```

```
Item Purchased_Coat
Item Purchased_Dress
Item Purchased_Gloves
Item Purchased_Handbag
Item Purchased_Hat
Item Purchased_Hoodie
Item Purchased_Jacket
Item Purchased_Jeans
Item Purchased_Jewelry
Item Purchased_Pants
Item Purchased_Sandals
Item Purchased_Scarf
Item Purchased_Shirt
Item Purchased_Shoes
Item Purchased_Shorts
Item Purchased_Skirt
Item Purchased_Sneakers
Item Purchased_Socks
Item Purchased_Sunglasses
Item Purchased_Sweater
Item Purchased_T-shirt
Category_Accessories
Category_Clothing
Category_Footwear
Category_Outerwear
Location_Alabama
Location_Alaska
Location_Arizona
```

전처리 후 남은 열들
뽑아보기

```
1 processed_df.to_csv("shopping_behavior_preprocessed.csv", index=False)
```

전처리가 끝난 데이터를 .csv 파일로 저장

발표 끝